



Chuyên đề

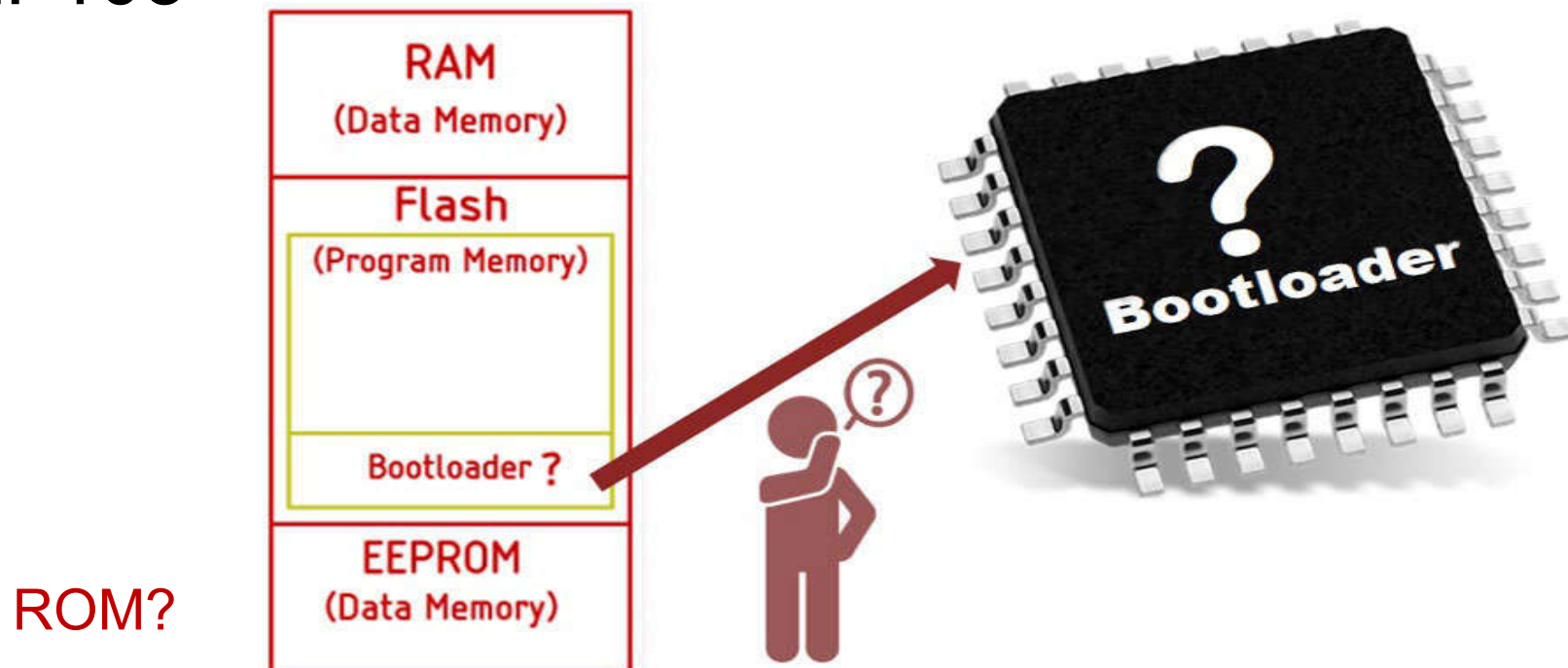
BOOTLOADER

GV: ThS. Nguyễn Đức Duy

Trường Điện – Điện tử
Đại học Bách Khoa Hà Nội

Nội dung

- Tổng quan về Bootloader
- Thiết kế và phân vùng bộ nhớ
- Bootloader trên chip STM32F1
- Triển khai Bootloader theo các dự án trên bộ Kit STM32F103



1 – Tổng quan về Bootloader

- Định nghĩa
- Cơ chế hoạt động
- Phân loại
- Mục đích sử dụng

1.1 – Định nghĩa

- Bootloader là công cụ mà các nhà sản xuất hoặc lập trình viên sinh ra, **được bật lên khi cần** để điều hướng các Firmware dùng đến khi khởi động
- Giúp người dùng nạp được code vào Flash mà không cần dùng mạch nạp chuyên dụng (như ST-Link, J-Link ...)
- Bản chất, Bootloader là một **chương trình phần mềm siêu nhỏ** cư trú tại địa chỉ thực thi lệnh đầu tiên (nếu được sử dụng) sau khi vi điều khiển khởi động hoặc Reset

1.2 – Cơ chế hoạt động

- Tùy thuộc vào nhân của dòng vi điều khiển sử dụng, người lập trình có những cách thức khác nhau để làm việc với chế độ Bootloader
- Mọi hệ thống Bootloader đều vận hành dựa trên 03 bước cốt lõi
 - **Bước 1:** Thiết lập chế độ Bootloader
 - **Bước 2:** Kiểm tra điều kiện
 - **Bước 3:** Điều hướng luồng
- Các loại nhân xử lý phổ biến trên thị trường hiện nay
 - Nhân ARM Cortex-M, nhân AVR, nhân Xtensa và RISC-V, nhân 8051 cũ và hiện đại, ...

Các loại nhân xử lý phổ biến

Họ nhân	Các dòng nhân tiêu biểu	Các chip đại diện
ARM Cortex-M	M0, M0+, M3, M4, M7, M23, M33	STM32 , nRF52832, Ti Tiva C, SAMD
AVR	AVR 8-bit, AVR 32-bit, AVR Dx	ATmega328P (Arduino), ATtiny, ATmega2560
RISC-V	RV32IMAC, RV32E	ESP32-C3 , GD32VF103, CH32V
Xtensa	L106, LX6, LX7	ESP32 , ESP8266
8051 (MCS-51)	Standard 8051, Enhanced 8051	STC8, Silicon Labs C8051, N76E003
PIC / dsPIC	PIC16, PIC18, PIC24, dsPIC33	PIC16F877A, dsPIC33 (chuyên điều khiển động cơ)
MSP430	MSP430X (16-bit)	MSP430G2553, MSP430FR series
MIPS	MIPS32 M4K, microAptiv	PIC32
RL78 / RX	RL78 (16-bit), RX (32-bit)	Renesas RL78, RX600

Bước 1: Thiết lập chế độ Bootloader

- Với các dòng cần **Fuse bits** như AVR
 - AVR sử dụng các ô nhớ đặc biệt gọi là Fuse bits để "cắm mốc" ranh giới cho Bootloader
 - Các bit như BOOTSZ0, BOOTSZ1 quy định kích thước, bit BOOTRST đưa chip vào chế độ Bootloader. Nếu không, mặc định chip sẽ chạy ứng dụng của người lập trình
- Với các dòng **không cần Fuse bits**
 - Người lập trình thiết kế dự án code, nạp vào địa chỉ Flash phù hợp cho chế độ **Custom Bootloader**. Fuse bits ở trên cũng đưa Chip vào chế độ này
 - Thiết lập yêu cầu theo nhà sản xuất cho chế độ **System Bootloader** (có sẵn theo Chip). Ví dụ với Blue Pill: Set Jumper BOOT0 = 1, BOOT1 = 0

Bước 2: Kiểm tra – Trigger

- **System Bootloader** (được nhà sản xuất **nạp sẵn** và không thể sửa đổi)
 - Cần một **cơ chế "cứng"** để CPU biết rằng lúc vừa khởi động, vào chế độ nạp code thay vì chạy code người dùng
 - Thường sử dụng các chân BOOT để kiểm tra
- Vào chế độ **Custom Bootloader**, để quyết định hướng đi tiếp theo cần quét một tín hiệu kích hoạt, **có nhiều cơ chế Trigger**
 - **Trigger nút nhấn**: Nhấn giữ một nút bấm trên mạch hoặc cắm ngoài khi cấp nguồn

Bước 2: Kiểm tra

- Chế độ Custom Bootloader
 - **Trigger thời gian:** Vừa bật nguồn, Bootloader sẽ chờ khoảng 1-2 giây. Nếu có dữ liệu gửi qua UART/USB, vào chế độ nạp, nếu không nhảy vào ứng dụng chính Application (Cách này thường thấy trên Arduino)
 - **Trigger phần mềm:** **Application** ghi một giá trị đặc biệt vào một ô nhớ RAM, sau đó thực hiện lệnh Reset mềm. Khi vừa khởi động lại, Bootloader kiểm tra ô nhớ RAM đó, nếu thấy giá trị đặc biệt, dừng lại để chờ nạp code. Hoặc ghi vào **thanh ghi dự phòng** (nên dung)
 - **Trigger qua giao thức:** **Bootloader tự mở** Bluetooth/UART, tự đợi lệnh qua giao thức truyền thông để yêu cầu chip vào chế độ DFU (Device Firmware Update)

Bước 2: Kiểm tra

- Lưu ý Trigger bằng giá trị đánh dấu trong RAM
 - Mặc dù dùng lệnh Reset mềm, ngay sau khi lệnh Reset được thực thi, chip nhảy vào hàm Reset_Handler trong file startup.c. Theo tiêu chuẩn lập trình C, hàm này có một nhiệm vụ bắt buộc: **Xóa sạch vùng nhớ .bss về 0**
 - **Giải pháp:** Biến đánh dấu trong RAM phải nằm trong vùng .noinit để tránh bị Reset_Handler xóa về 0 trong quá trình khởi tạo môi trường C
 - **Cách thực hiện:** Khai báo với thuộc tính đặc biệt (ví dụ: `__attribute__((section(".noinit")))` để trình biên dịch không xóa nó khi Reset

Bước 2: Kiểm tra

- Lưu ý Trigger bằng thanh ghi dự phòng BKP
 - Ghi giá trị vào một ô thanh ghi phần cứng chuyên biệt (Ví dụ: `BKP->DR1`). Ô thanh ghi này nằm trong một vùng an toàn gọi là miền năng lượng dự phòng (RTC Domain)
 - **Soft Reset:** An toàn 100%
 - **Hard Reset:** An toàn 100%. Nút Reset vật lý trên board mạch không thể xóa được dữ liệu bên trong thanh ghi BKP
 - **Rút nguồn điện:** Tiếp tục lưu trữ nếu mạch được thiết kế có một viên pin đồng hồ (CR2032) cấp điện cho chân VBAT

Bước 3: Điều hướng

- Chế độ **cập nhật Firmware**
 - Nếu Trigger thỏa mãn (Custom Bootloader) hoặc chế độ System Bootloader, dừng lại để nhận dữ liệu Firmware mới qua các cổng truyền thông
- Chế độ **chạy ứng dụng**
 - Nếu không có Trigger, Custom Bootloader thực hiện lệnh "nhảy" để bàn giao quyền kiểm soát hoàn toàn cho Application. Với STM32F103, lệnh nhảy này nhảy tới địa chỉ Reset_Handler của Application sau khi đã nạp lại giá trị Stack Pointer

So sánh Bootloader trên các loại nhân

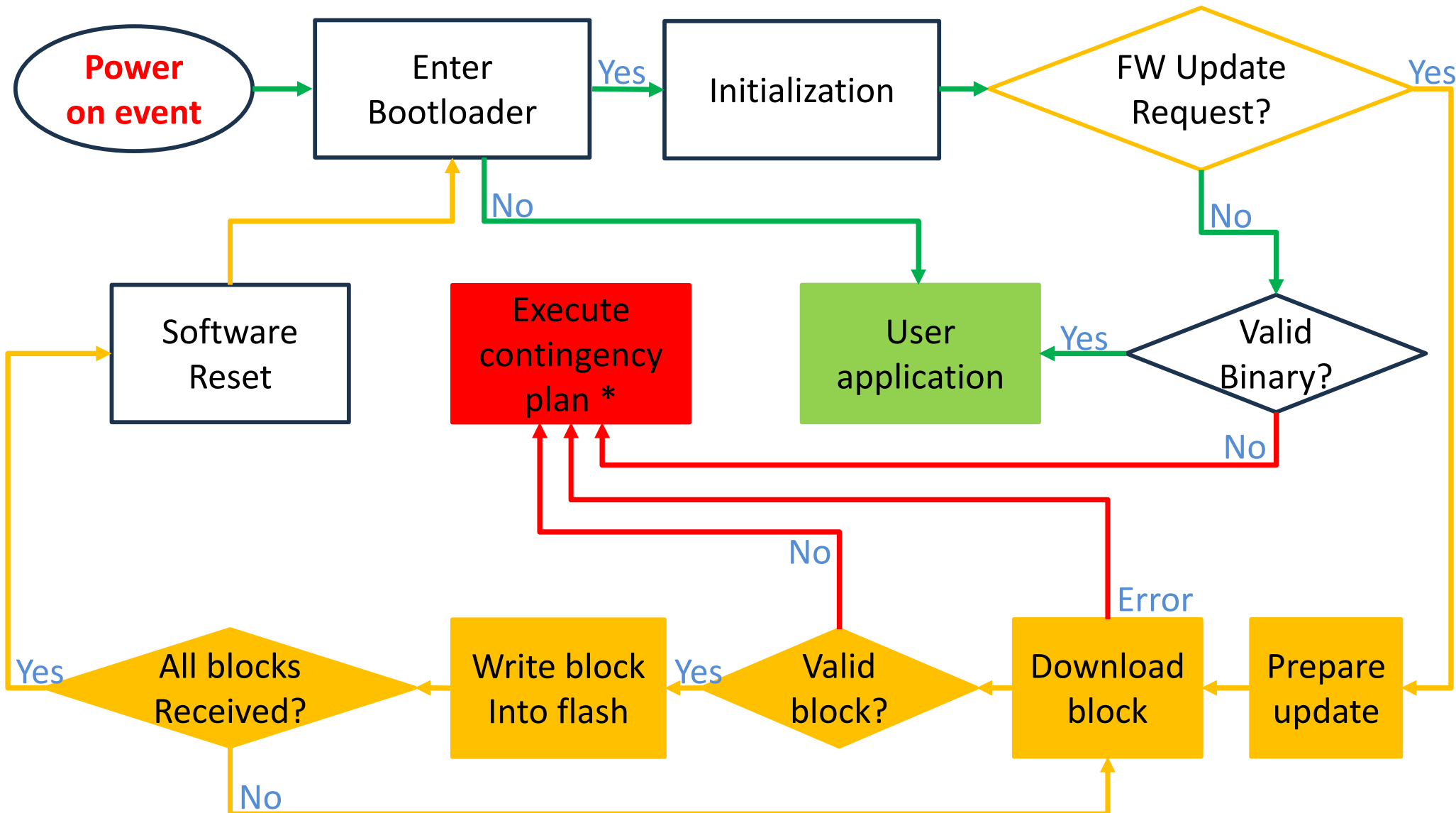
Nhân xử lý	Có System Bootloader?	Cần thiết lập Fuse/Config?	Có thể Custom Bootloader?
ARM Cortex-M	Có (Đa số)	Không	Rất tốt (Sử dụng VTOR)
AVR	Không	Bắt buộc	Có (Hạn chế về dung lượng)
ESP32 (Xtensa)	Có	Không	Có (Hỗ trợ 2-stage bootloader)
RISC-V	Tùy hãng (ESP: Có)	Tùy hãng	Có
MSP430	Có (BSL)	Không	Có
8051 (Cũ)	Không	Không hỗ trợ	Rất khó

So sánh Custom Bootloader AVR và ARM ^{Ø.14}

Đặc điểm	Dòng AVR	Dòng ARM (STM32, Tiva...)
Vị trí Bootloader	Thường nằm ở cuối Flash	Thường nằm ở đầu Flash
Cơ chế nhảy	Dùng Fuse bits (BOOTRST) để ép CPU nhảy tới vùng Bootloader, việc rẽ nhánh do của người lập trình điều khiển	CPU mặc định nhảy vào đầu Flash, việc rẽ nhánh do Code của người lập trình điều khiển
Cấu hình	Dùng mạch nạp chuyên dụng để set Fuse bits một lần duy nhất	Cấu hình linh hoạt thông qua phần mềm (Linker Script và Vector Table)

1.2 – Cơ chế hoạt động

Ø.15



* Kế hoạch dự phòng

1.3 – Phân loại

- Có thể chia thành 2 loại
 - System Bootloader
 - Custom Bootloader
- In-Application Programming là 1 trường hợp đặc biệt của Custom Bootloader là

1.3 – Phân loại

■ System Bootloader

- Là một chương trình đặc biệt **được nạp sẵn vào chip** ngay từ khi sản xuất, không thể thay đổi
- Được kích hoạt khi thiết lập theo yêu cầu của nsx
- Vai trò như một hệ điều hành cứu hộ **nằm trong vùng nhớ ROM**
- Cho phép nạp Firmware mới vào bộ nhớ Flash thông qua các giao diện ngoại vi như UART, USB, CAN, I2C, SPI mà không cần dùng mạch nạp JTAG/SWD chuyên dụng
- Dòng AVR, 8051 cũ ... không có System Bootloader trong ROM giống như các dòng chip 32-bits hiện đại

1.3 – Phân loại

- Custom Bootloader
 - Chương trình do chính người lập trình viết và nạp vào vùng Flash thích hợp
 - Tự định nghĩa cách cập nhật rất linh hoạt (ví dụ: lấy code từ thẻ nhớ SD, qua Bluetooth, hoặc mã hóa dữ liệu để bảo mật...)
- Custom Bootloader vs System Bootloader
 - Nếu System Bootloader là cứu hộ mặc định của nhà sản xuất, Custom Bootloader chính là điều phối do người lập trình tự thiết kế để điều hành Flash
 - Đây là chương trình chạy ở tầng User Flash, có toàn quyền quyết định việc thực thi hoặc cập nhật ứng dụng mà không cần thiết lập theo yêu cầu từ đầu như System Bootloader

1.3 – Phân loại

- In-Application Programming
 - Thực chất là một kỹ thuật nâng cao của Custom Bootloader
 - **Cơ chế:** Chương trình Application đang chạy bình thường vẫn có thể tự nhận dữ liệu mới và ghi vào một vùng Flash khác (thường là vùng tạm). Sau đó chip tự Reset để Bootloader trao đổi bản cũ và bản mới
 - **Ứng dụng:** Cập nhật từ xa (OTA - Over The Air) cho các thiết bị IoT mà không cần thao tác trực tiếp trên mạch

1.4 – Mục đích sử dụng Custom Boot

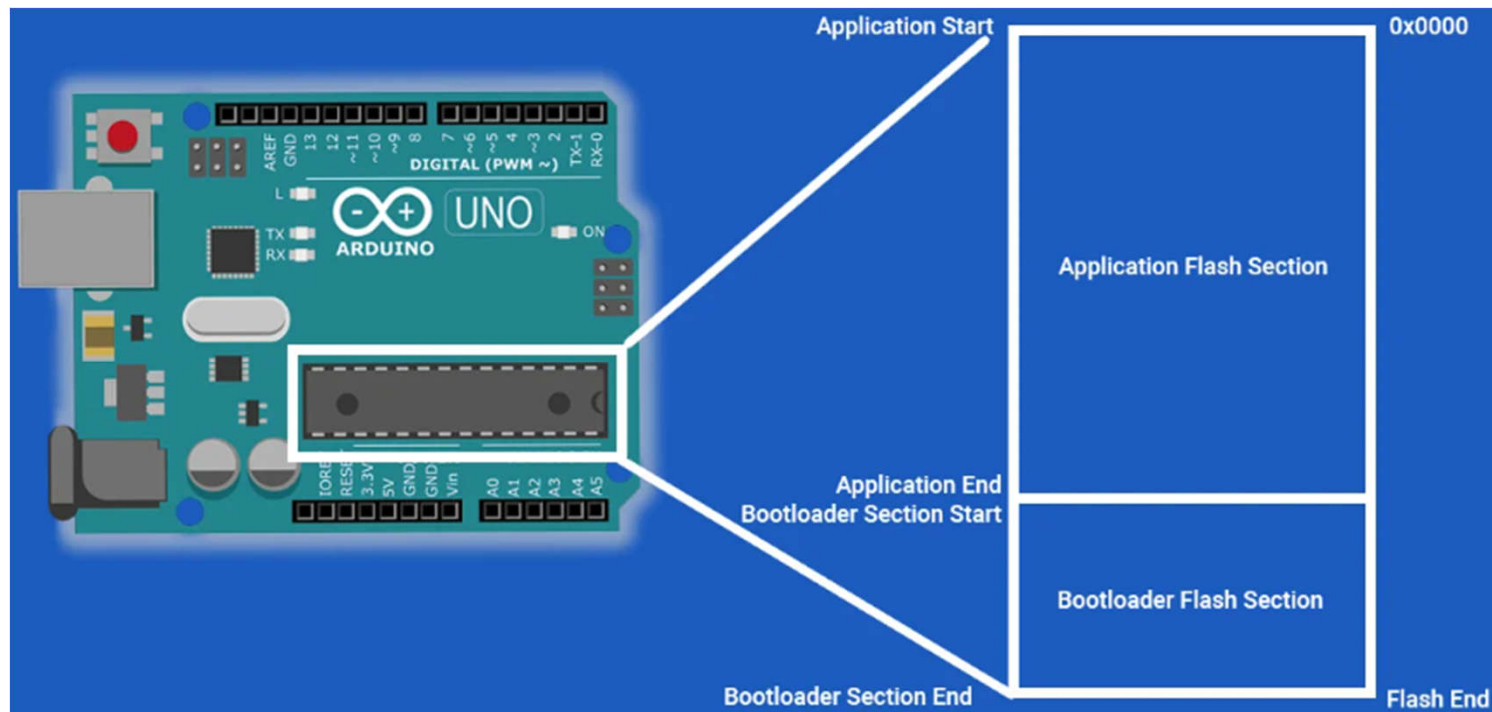
- Cập nhật từ xa/không cần mạch nạp
 - Qua Bluetooth, thẻ nhớ SD, Wifi hoặc các cổng truyền thông thay vì phải sử dụng mạch nạp
- Bảo mật
 - Có thể mã hóa file Firmware. Chỉ Custom Bootloader mới có chìa khóa để giải mã và ghi vào Flash, ngăn chặn việc bị đánh cắp code
- Tính toàn vẹn
 - Dùng mã CRC để kiểm tra file nạp có bị lỗi bit nào không trước khi cho phép chạy
- Brick (Chip mất khả năng hoạt động)
 - Xây dựng hệ thống cập nhật Firmware có khả năng dự phòng mà System Bootloader không làm được

1.4 – Mục đích sử dụng Custom Boot

Tiêu chí	Bootloader cơ bản – <u>sơ khởi</u>	Bootloader Dual-Bank – <u>ổn định</u>	Bootloader IoT (OTA) – <u>hiện đại</u>
Mục tiêu	Tiện lợi (không cần mạch nạp)	An toàn (chống Brick chip)	Từ xa và Tự động
Rủi ro lỗi nạp	Chip bị treo (Brick)	Tự động quay về bản cũ (Rollback)	Tự động khôi phục qua mạng
Vùng nhớ	1 vùng Application duy nhất	2 vùng Bank A và Bank B	Phức hợp (Metadata + Multi-Bank)
Ứng dụng	Học tập, nạp code tại bàn	Thiết bị công nghiệp, y tế	Thiết bị IoT, nhà thông minh

2 – Thiết kế và phân vùng bộ nhớ

- Định nghĩa
- Phân vùng bộ nhớ
- Lưu ý quan trọng



2.1 – Định nghĩa

- Phân vùng bộ nhớ là việc chia bộ nhớ Flash vật lý thành các khu vực logic riêng biệt, mỗi vùng đảm nhận một vai trò cụ thể
- Chỉ áp dụng với Custom Bootloader, sau đây gọi tắt là Bootloader
- Mục đích
 - Ngăn chặn việc Bootloader tự xóa chính nó hoặc ghi đè dữ liệu quan trọng, tránh tình trạng **brick** chip
- Tính cố định
 - Các phân vùng này được xác định rõ ràng về địa chỉ bắt đầu và kích thước ngay từ khâu thiết kế hệ thống

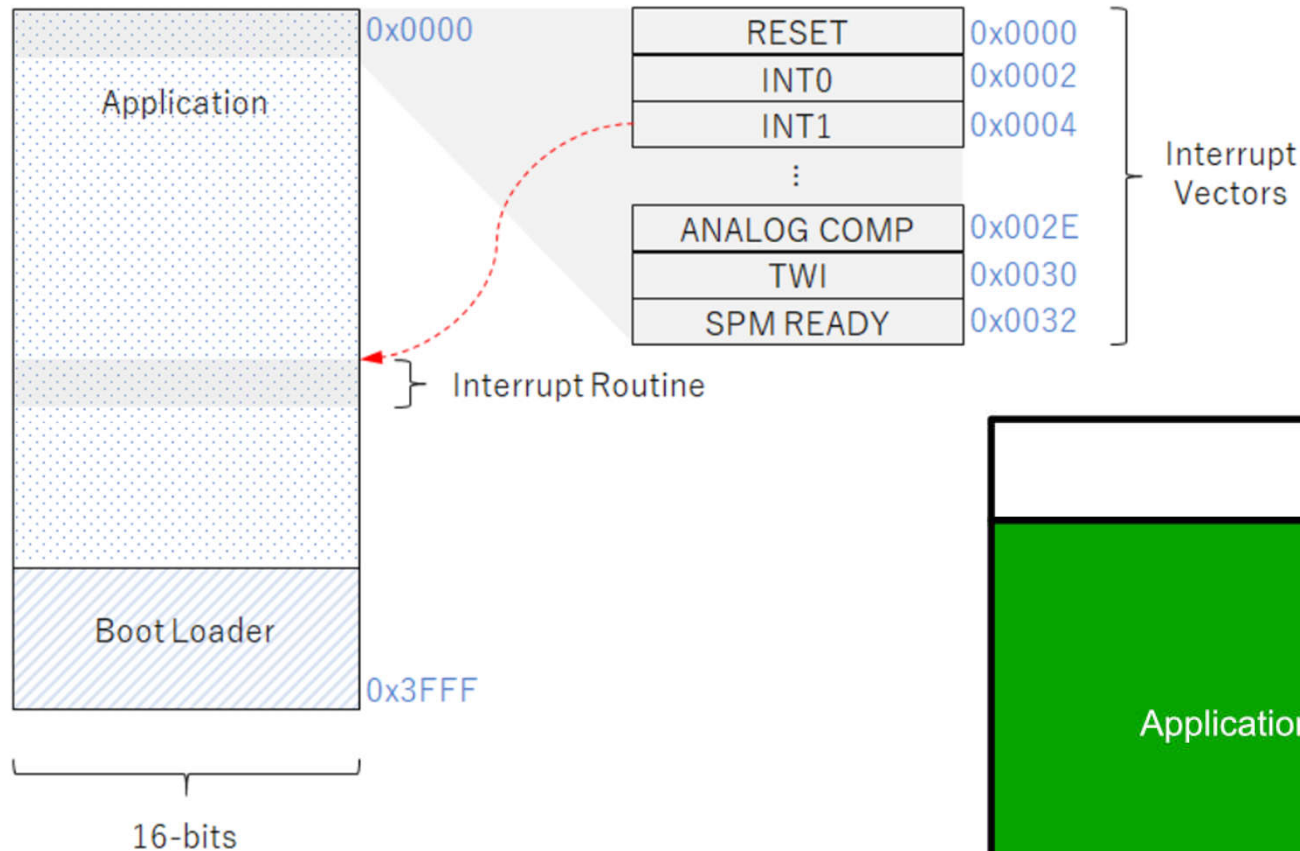
2.2 – Phân vùng bộ nhớ

- Cấu trúc vật lý của Flash (Page hay Sector) quyết định trực tiếp chiến lược phân vùng
 - Cấu trúc **Sector** như STM32F4,....
 - **Đặc điểm**: Flash chia thành **các Sector** với kích thước **khác nhau** (ví dụ: 16KB, 64KB, 128KB, 256KB)
 - **Phân vùng**: Phải thiết kế khéo léo, thường đặt Bootloader vào **các Sector nhỏ đầu tiên** (như Sector 0 - 16KB) để tránh lãng phí các Sector lớn (128KB, 256KB)

2.2 – Phân vùng bộ nhớ

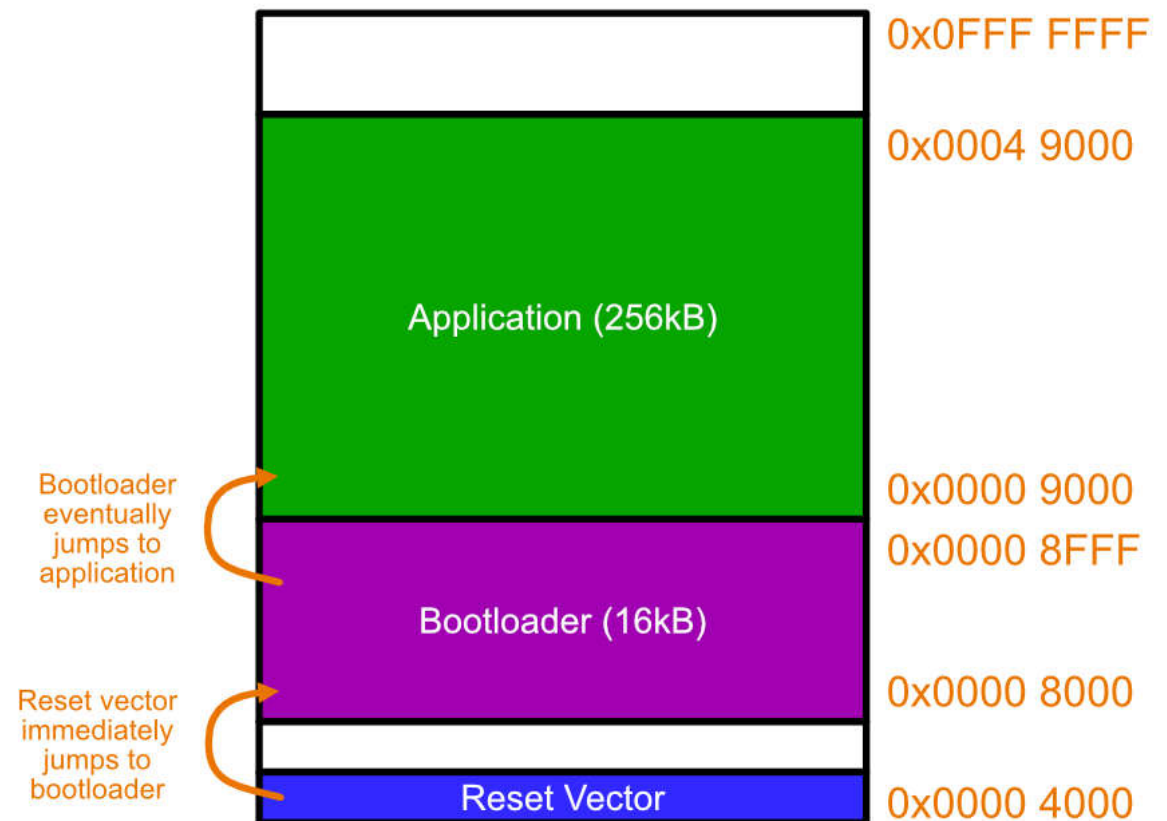
- Cấu trúc **Page** như AVR, STM32F1, ...
 - **Đặc điểm**: Toàn bộ Flash được chia thành **các trang (Page)** có kích thước **bằng nhau** (thường là 1KB hoặc 2KB)
 - **Phân vùng**: Dễ dàng tính toán, dành ra một số lượng **Page nhất định** ở đầu hoặc cuối Flash cho Bootloader
 - Toàn bộ 64KB Flash của **Blue Pill** (dòng Medium Density) được chia thành 64 trang (Page), mỗi trang có kích thước cố định là 1KB
 - Rất linh hoạt và tiết kiệm. Ta có thể cấp phát chính xác 8 Page (8KB) cho Bootloader mà không lãng phí hàng chục KB như cấu trúc Sector của F4

2.2 – Phân vùng bộ nhớ



Bản đồ bộ nhớ Flash
Chip **ATmega328P** dùng
trong Kit Arduino Uno

Bản đồ bộ nhớ Flash
Chip Texas Instruments
Tiva C (**TM4C123**)



2.2 – Phân vùng bộ nhớ

- Cơ chế Dual-Bank (Vi điều khiển đòi hỏi cao)
 - **Đặc điểm:** Flash được chia làm **hai nửa vật lý độc lập** (Bank 1 và Bank 2)
 - **Phân vùng:** Cho phép thực hiện cơ chế A/B Update: **Một Bank chạy ứng dụng cũ, một Bank nhận dữ liệu mới** đồng thời, giúp hệ thống luôn có bản dự phòng an toàn
- Áp dụng Dual-Bank cho Flash chỉ có 1 Bank
 - Với các dòng chip chỉ có **1 khối Flash** duy nhất (Single Bank) nhưng **dung lượng lớn**
 - **Không thể vừa chạy code vừa ghi vào Flash cùng một lúc** trên cùng một chip. Thông thường, Bootloader sẽ phải dừng hẳn việc chạy Application để thực hiện nạp vào Bank trống

2.2 – Phân vùng bộ nhớ

Tiêu chí	Người lập trình tự chia (Logic)	Nhà sản xuất chia sẵn (Hardware)
Áp dụng	Mọi dòng chip có Flash đủ lớn	Chỉ các dòng cao cấp, đắt tiền
Độ an toàn	Cao (có bản dự phòng để Rollback)	Cực cao (chống Brick tuyệt đối)
Cơ chế nạp	Phải Reset để cập nhật. Thường nạp trong Bootloader	Có thể cập nhật "nóng" (Live Update) trong khi App đang thực thi
Độ phức tạp	Thấp, dễ triển khai trên mọi chip	Cao, đòi hỏi phần cứng hỗ trợ chuyên dụng

Các thành phần chính

- Bootloader Region
 - Chứa mã nguồn thực thi của Bootloader, driver ngoại vi (UART ...) và logic kiểm tra điều kiện nhảy
- Metadata / Config
 - Vùng nhỏ chứa thông tin quản lý: Magic Number (xác nhận code hợp lệ), Version (phiên bản), và mã kiểm tra lỗi CRC
- Application Region
 - Vùng lớn nhất dành cho chương trình chính
 - Single Bank
 - Dual-Bank (dùng thanh ghi hệ thống để "tráo đổi" địa chỉ 2 Bank)

2.3 – Các lưu ý quan trọng

- Quản lý Bảng Vector ngắt
 - **Vấn đề:** Mặc định bảng Vector nằm ở đầu Flash (0x0800 0000 trên STM32 hoặc 0x0000 trên AVR,...)
 - **Giải pháp**
 - Khi nhảy sang App (ở địa chỉ khác), người lập trình buộc phải cập nhật lại thanh ghi vector (như `SCB->VTOR` trên ARM) trở về địa chỉ bắt đầu của App
 - Nếu không, khi App đang chạy mà có ngắt, CPU sẽ nhảy nhầm về vùng code của Bootloader và gây HardFault
- Dọn dẹp ngoại vi trước khi Nhảy
 - Bootloader thường sử dụng UART, Timer hoặc Clock để phục vụ quá trình nạp
 - Trước khi thực hiện lệnh Jump, phải đưa các ngoại vi này về trạng thái mặc định (`De-init`)

2.3 – Các lưu ý quan trọng

- Dọn dẹp ngoại vi trước khi Nhảy
 - **Quan trọng nhất:** Phải vô hiệu hóa ngắt toàn cục (`__disable_irq()` trong ARM) để đảm bảo không có ngắt nào xảy ra trong khoảng khắc CPU đang chuyển giao quyền điều khiển
- Cấu hình Fuse bits, file Linker Script (.ld), bảng phân vùng (Partition Table) ... chính xác
 - File Linker, Partition Table hoặc Fuse bits là nơi định nghĩa "mảnh đất" cho từng chương trình
 - **Bootloader:** Phải giới hạn LENGTH đúng bằng kích thước Sector/Page dự định để tránh code phình to đè vào vùng App
 - **Application:** Phải thay đổi ORIGIN tới địa chỉ bắt đầu mới (sau vùng Bootloader/Metadata)

2.3 – Các lưu ý quan trọng

- Kiểm tra tính toàn vẹn
 - Không bao giờ tin tưởng tuyệt đối vào đường truyền UART/OTA... do nhiều là không thể tránh khỏi
 - **Giải pháp:** Luôn sử dụng Checksum hoặc CRC để kiểm tra file App sau khi nạp xong
 - **Cơ chế:** Chỉ khi mã CRC tính toán khớp với mã lưu trong Metadata, Bootloader mới cho phép thực hiện lệnh Jump

3 – Bootloader trên chip STM32F1

- Phân chia vùng nhớ
- Chiến lược phân vùng
- Các lưu ý trước khi triển khai
- Cấu hình trong Projects

3.1 – Phân chia vùng nhớ

- Dòng F103 Medium-density (Blue Pill) có cấu trúc Page, không phải Sector. Toàn bộ 64KB Flash được chia thành 64 trang, mỗi trang cố định 1KB (1024 bytes) như bảng sau

Phân vùng	Địa chỉ bắt đầu	Kích thước	Số trang (Page)	Mục đích sử dụng
Bootloader	0x0800 0000	8 KB	Page 0 - 7	Chứa code Bootloader, Driver UART/Flash
Metadata	0x0800 2000	1 KB	Page 8	Lưu Flag cập nhật, Version, mã CRC
Application	0x0800 2400	55 KB	Page 9 - 63	Vùng chứa chương trình chính của người dùng

3.2 – Chiến lược phân vùng

- Bootloader Region (8KB đầu tiên)
 - **Vị trí:** Luôn bắt đầu từ **0x0800 0000**
 - **Nhiệm vụ:** Đây là vùng **CPU nhảy vào đầu tiên** sau khi Reset. Nó chứa các driver cơ bản (nhận file **.bin** qua UART, Flash, GPIO) và logic kiểm tra điều kiện Jump
 - **Lưu ý:** Đối với Blue Pill, 8KB (8 trang) là khoảng trống đủ để chứa thư viện HAL, Driver UART và logic ghi Flash

3.2 – Chiến lược phân vùng

- Metadata/Signature Region (Page 8)
 - **Vị trí:** Địa chỉ **0x0800 2000**
 - **Nhiệm vụ:** Đóng vai trò là biên giới và sổ nhật ký. Lưu trữ các dấu vết như
 - *Magic Number*: Một con số đặc biệt để xác nhận vùng App đã được nạp code hợp lệ chưa hay cần cập nhật lại (tránh nhảy vào vùng trống)
 - *App Version*: Để Bootloader không nạp lại bản cũ
 - *Checksum (CRC)*: Mã kiểm tra lỗi của toàn bộ vùng App trước khi thực thi

3.2 – Chiến lược phân vùng

- Application Region (Page 9 - 63 vùng lớn nhất)
 - **Vị trí:** Thường bắt đầu từ **0x0800 2400** hoặc **0x0800 3000**
 - **Nhiệm vụ:** Chứa toàn bộ Firmware chương trình các tính năng chính của người lập trình
 - Blue Pill có dung lượng Flash khiêm tốn (64KB). Việc chia phân vùng không hợp lý (dành quá nhiều cho Bootloader) sẽ làm thu hẹp không gian cho ứng dụng chính
 - Phân chia theo đúng kích thước trang (1KB) giúp tối ưu hóa việc xóa (Erase) và ghi (Write) Flash, vì lệnh xóa Flash trên F1 bắt buộc **thực hiện theo từng trang**

3.3 – Các lưu ý trước khi triển khai

- Chuẩn bị trước khi nhảy từ Bootloader sang App
 - Kiểm tra tính hợp lệ của vùng nhớ
 - Đọc giá trị tại địa chỉ đầu tiên của App (0x0800 2400). Giá trị này chính là Top of Stack (MSP) của Application
 - Nó phải nằm trong vùng RAM của chip (0x2000 0000 đến 0x2000 5000 đối với 20KB RAM của F103)
 - Hủy cấu hình ngoại vi
 - Tắt các bộ ngoại vi đã dùng trong Bootloader (hủy khởi tạo UART, tắt Timer, tắt Clock liên quan)
 - Vô hiệu hóa ngắt toàn cục
 - Gọi lệnh `__disable_irq()` để ngăn chặn bất kỳ ngắt nào xảy ra trong tích tắc chuyển giao lỗi

3.3 – Các lưu ý trước khi triển khai

- Kiểm tra tính toàn vẹn và quản lý lỗi
 - **Nguyên cơ:** Quá trình nạp Firmware qua UART bị ngắt quãng, vùng Application chỉ chứa một nửa dữ liệu. Nếu Bootloader vẫn nhảy vào địa chỉ này, chip sẽ rơi vào trạng thái ngắt HardFault liên tục
 - **Giải pháp:** Ứng dụng trang Metadata (Page 8)
 - *Ghi nhận trạng thái:* Khi ứng dụng chính truyền xong, tính toán mã CRC32 của toàn bộ vùng App (từ 0x0800 2400 đến hết dung lượng file nhận được), ghi giá trị CRC cùng một thẻ đánh dấu trạng thái thành công (Magic Number, ví dụ: 0x5A5A5A5A) vào vùng Metadata
 - *Xác thực khi Boot:* Mỗi khi khởi động, Bootloader đọc Page 8. Kiểm tra không khớp Magic Number hoặc tính lại CRC vùng App thấy sai lệch, Bootloader sẽ từ chối nhảy và tự động chuyển sang chế độ chờ nạp lại Firmware mới qua UART

3.4 – Cấu hình trong Projects

- Chỉnh sửa file .ld (Linker Script)

- Đối với Project Bootloader

Giới hạn Flash ở 8KB để trình biên dịch báo lỗi nếu code Bootloader phình quá to.

```
/* file STM32F103C8_BOOTLOADER.ld */
```

```
MEMORY
```

```
{
```

```
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 8K
```

```
    RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 20K
```

```
}
```


3.4 – Cấu hình trong Projects

- Chỉnh sửa file .ld (Linker Script)

- Đối với Project Application

Dời địa chỉ bắt đầu (ORIGIN) và tính lại độ dài còn lại (LENGTH).

```
/* file STM32F103C8_APP.ld */  
MEMORY  
{  
/* Sau vùng Bootloader (8K) và Metadata (1K) */  
    FLASH (rx): ORIGIN = 0x08002400, LENGTH = 55K  
/* 64K - 8K(Bootloader) - 1K(Metadata) */  
    RAM (xrw): ORIGIN = 0x20000000, LENGTH = 20K  
}
```

3.4 – Cấu hình trong Projects

■ Chỉnh sửa file main

– Đối với Project Bootloader

- Hàm nhảy từ Bootloader đến Application

`/* Cụ thể theo trong từng dự án */`

- Vô hiệu hóa ngắt toàn cục trước khi nhảy

`__disable_irq();`

- User code

– Đối với Project Application

- Dời bảng Vector

`SCB->VTOR = 0x08002400;`

- Bật ngắt toàn cục

`__enable_irq();`

- User code

3.4 – Cấu hình trong Projects

- App không được lần sang các vùng nhớ cuối
 - System Memory là vùng nhớ ROM chỉ đọc, được STM nạp sẵn từ nhà máy, chứa System Bootloader gốc của hãng
 - Option Bytes chứa các byte cấu hình phần cứng cấp thấp

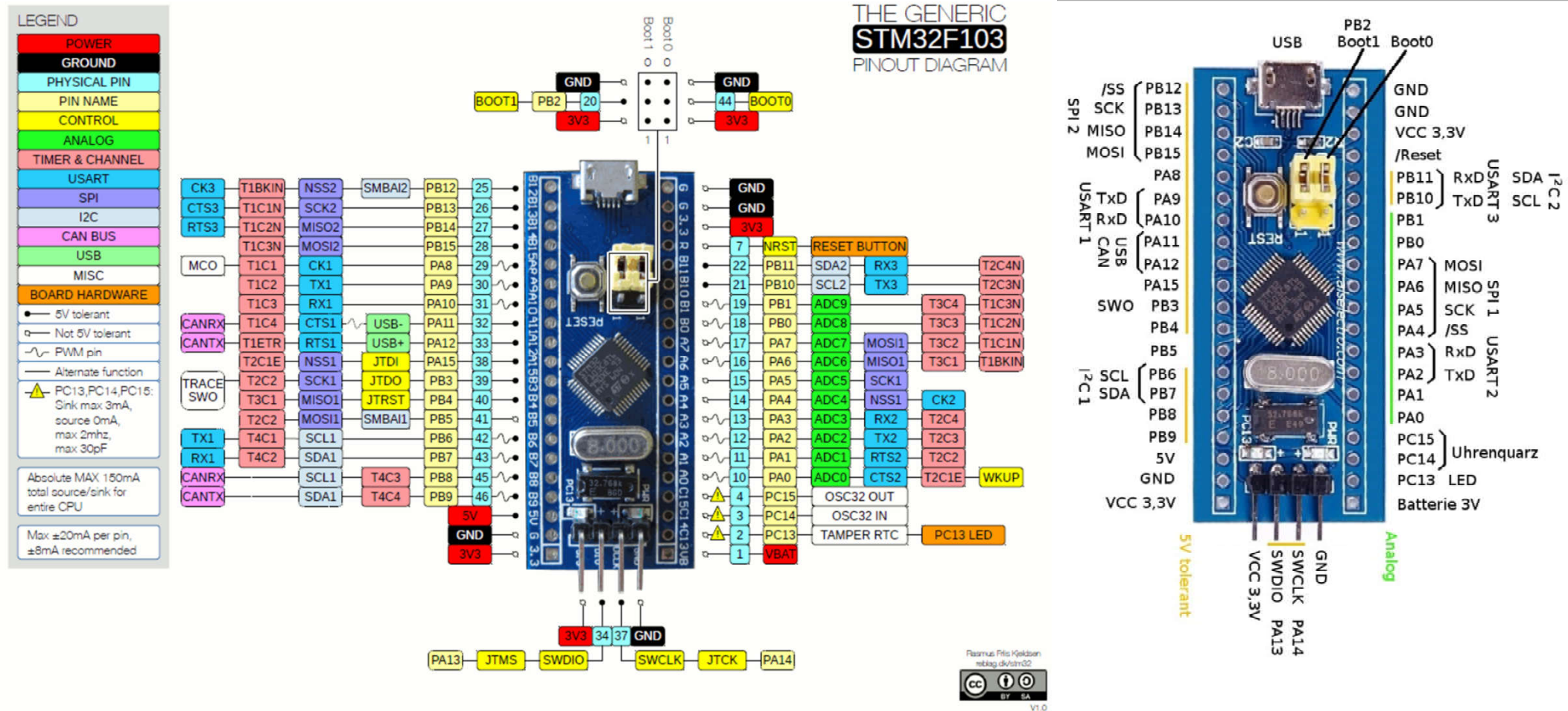
Khối	Phân vùng	Kích thước	Ký hiệu	Địa chỉ bắt đầu	Địa chỉ kết thúc
Main Memory	Page 0	1 KB	Page 0	0x0800 0000	0x0800 03FF
	Page 1	1 KB	Page 1	0x0800 0400	0x0800 07FF
	Page 2	1 KB	Page 2	0x0800 0800	0x0800 0BFF
	...	...	...	...	...
	Page 63	1 KB	Page 63	0x0800 FC00	0x0800 FFFF
	(Chỉ có trên bản CB/128KB)	1 KB	Page 64-127	0x0801 0000	0x0801 FFFF
Information Block	System Memory	2 KB	-	0x1FFF F000	0x1FFF F7FF
	Option Bytes	16 Bytes	-	0x1FFF F800	0x1FFF F80F
Flash Interface (Thanh ghi điều khiển)	Flash Registers	-	-	0x4002 2000	0x4002 203F

4 – Triển khai Bootloader theo các dự án ^{Ø.44}

- Tổng quan về Bo mạch thực hành
- Dự án 1: Cú nhảy đầu tiên
- Dự án 2: Bootloader có điều kiện
- Dự án 3: Cập nhật Firmware qua UART
- Dự án 4: Bootloader an toàn và Bảo mật
- Dự án 5: Bootloader Dual-Bank

4.1 – Tổng quan về bo mạch thực hành

Ø.45



[Tổng quan về bo mạch thực hành.docx](#)

4.2 – Dự án 1: Cú nhảy đầu tiên

- **Mục tiêu:** Hiểu cách dời địa chỉ và thực hiện lệnh Jump đơn giản nhất
- **Kịch bản**
 - Khi cấp nguồn, Bootloader nháy nhanh LED0 ngoài 3 lần, sau đó tự động nhảy sang chạy App – nháy chậm LED1 ngoài 3 lần
 - *Địa chỉ 0x0800 2400 (Offset 0):* Chứa giá trị Stack Pointer
 - *Địa chỉ tiếp theo (Offset +4):* Chứa địa chỉ của Reset Handler
- **Kiến thức trọng tâm**
 - Cấu hình Linker Script (.ld) cho Application
 - Sử dụng con trỏ hàm để gọi Reset Handler của App
- **Kết quả:** LED0 nháy (Bootloader chạy) rồi LED1 nháy (App chạy)

4.3 – Dự án 2: Bootloader có điều kiện

- **Mục tiêu:** Kiểm soát việc khởi động bằng nút nhấn vật lý
- **Kịch bản**
 - Nếu **nhấn giữ nút nhấn ngoài (PA0)** khi bấm Reset: Chip ở lại Bootloader (LED0 sáng đứng) để chờ lệnh (LED2 nháy)
 - Nếu không nhấn nút: Chip nhảy thẳng vào App như Dự án 1
- **Kiến thức trọng tâm:** Đọc trạng thái GPIO sớm và xây dựng logic rẽ nhánh cho quá trình Boot
- **Kết quả:** Không nạp code liên tục, chỉ khi nào muốn "giữ" chip lại mới cần dùng nút nhấn

4.4 – Dự án 3: Cập nhật Firmware

- **Mục tiêu:** Xây dựng một **công cụ nạp** giống như ST-Link **qua cổng Serial**
- **Kịch bản**
 - Khi ở chế độ Bootloader (do nhấn giữ nút PA0), chip chờ nhận dữ liệu từ máy tính qua UART
 - Gửi file **.bin** xuống, Bootloader sẽ xóa các Page tương ứng và ghi đè bản mới vào hoặc ghi vào vùng nhớ trống
- **Kiến thức trọng tâm**
 - HAL Flash Driver: Các hàm `HAL_FLASH_Unlock`, `HAL_FLASHEx_Erase`, `HAL_FLASH_Program`.
 - Giao thức UART nhận dữ liệu
 - Xử lý file nhị phân

4.4 – Dự án 3: Cập nhật Firmware

- **Kết quả:** Có thể cập nhật Firmware mới mà không cần cắm mạch nạp ST-Link, chỉ cần USB-to-UART
- Hướng dẫn chi tiết tham khảo tại: [Dự án 3.docx](#)

4.5 – Dự án 4: An toàn và Bảo mật

- **Mục tiêu:** Đảm bảo chip không bao giờ bị "brick" và code không bị đọc trộm
- **Kịch bản**
 - Check CRC: Trước khi Jump, Bootloader tính toán mã kiểm tra của vùng App. Nếu sai (do nạp lỗi), không Jump để tránh treo chip
 - Mã hóa: File .bin trên máy tính được mã hóa. Bootloader nhận về phải giải mã rồi mới ghi vào Flash
 - Go-to-Bootloader Command: App đang chạy, nếu gửi một lệnh từ máy tính, App tự nhảy về Bootloader mà không cần bấm nút Reset

4.5 – Dự án 4: An toàn và Bảo mật

- **Kiến thức trọng tâm**
 - Thuật toán Checksum/CRC
 - Thư viện mã hóa cơ bản (AES hoặc XOR đơn giản)
 - Kỹ thuật chuyển đổi ngược từ App về Bootloader
- Hướng dẫn chi tiết tham khảo tại: [Dự án 4.docx](#)

4.6 – Dự án 5: Bootloader Dual-Bank

- **Mục tiêu:** Tự xây dựng hệ thống cập nhật Firmware có khả năng dự phòng. Đảm bảo thiết bị luôn có một bản Firmware hoạt động tốt, loại bỏ hoàn toàn rủi ro "brick" chip khi mất điện hoặc lỗi file lúc đang nạp
- **Kịch bản**
 - Bộ nhớ Flash được chia làm 2 vùng: Bank A (vùng đang chạy) và Bank B (vùng chờ nạp)
 - Khi ở chế độ Bootloader, chip nhận file .bin mới qua UART và ghi vào Bank đang trống (Bank B)
 - Sau khi nạp xong, Bootloader kiểm tra tính toàn vẹn của bản mới. Nếu ổn, thay đổi cờ (Flag) trong Flash để trở địa chỉ Jump sang Bank mới ở lần khởi động sau

4.6 – Dự án 5: Bootloader Dual-Bank

■ Kịch bản

- Nếu bản mới lỗi, hệ thống tự động quay về chạy bản cũ ở Bank còn lại

■ Kiến thức trọng tâm

- Phân vùng Flash phức hợp: Chia đôi 55KB Flash còn lại của Blue Pill thành 2 nửa (Bank A bắt đầu từ 0x0800 2400 và Bank B bắt đầu từ 0x0800 9000)
- Quản lý Metadata (Page 8 - 0x0800 2000): Lưu trữ trạng thái hệ thống (Đang chạy Bank nào? Bản nạp mới có thành công không?)
- Logic Jump linh hoạt: Bootloader đọc Flag từ Flash để quyết định nhảy vào 0x0800 2400 hay 0x0800 9000

4.6 – Dự án 5: Bootloader Dual-Bank

- **Kiến thức trọng tâm**
 - Cấu hình Linker Script kép: Hiểu cách biên dịch 2 phiên bản Application cho 2 vùng địa chỉ khác nhau
- **Kết quả:** Có thể cập nhật Firmware v2 vào Bank B trong khi Firmware v1 vẫn an toàn ở Bank A. Nếu quá trình nạp v2 bị ngắt quãng (rút dây UART), chip vẫn hoạt động bình thường với Firmware v1, không bao giờ bị treo cứng
- Hướng dẫn chi tiết tham khảo tại: [Dự án 5.docx](#)

GV: ThS. Nguyễn Đức Duy

Trường Điện – Điện tử
Đại học Bách Khoa Hà Nội

